

ClashArena — Architecture Characteristics

Which characteristics from the course list matter for our system, and what we're aiming for

Shattyk Kuziyeva (913168)

Daniil Glazunov (912512)

1. How we picked these

The course gives a fixed list of twelve characteristics (slide 18, “Our architecture characteristics”): partitioning type, simplicity, modularity, maintainability, testability, deployability, evolvability, responsiveness, scalability, elasticity, fault tolerance, and overall cost. We're not allowed to just pick our favourites — the rule from lecture 2 is that a characteristic only counts if it (1) isn't about the domain itself, (2) actually pushes the design in some direction, and (3) really matters for ClashArena to succeed, not just “would be nice.”

We also tried to keep the list short on purpose. Every characteristic you add is more stuff to build and defend, and “shoot for the least worst architecture, not the best one” was said more than once in the slides, so we took that seriously rather than just picking all twelve because they all sound relevant on paper.

Below, every characteristic we kept is tied back to a specific line in the kata. We numbered the kata (R1–R10 for requirements, U1–U11 for users, C1–C8 for additional context) specifically so we could point at exact items instead of hand-waving — the updated, numbered kata is attached after this document.

2. What we kept and what we dropped

#	Characteristic	In or out	Why
1	Partitioning type	In	C6 basically spells this out for us — different people own different domains
2	Simplicity	In	The whole point of building a monolith at all
3	Modularity	In	Follows from C6/C7
4	Maintainability	Out	Too close to Evolvability/Testability for a project this short-lived

5	Testability	In	R5 and R9 both hinge on results being verifiable
6	Deployability	In	C7 — has to run in containers, locally and in the cloud
7	Evolvability	In	C6 again — independent evolution by different team members
8	Responsiveness	In	R6/R7, C4 — near-real-time standings, global read traffic
9	Scalability	In	U8 — 10x account growth expected over 24 months
10	Elasticity	In	U10/C1 — spectator bursts, then back to baseline
11	Fault tolerance	In	C2/C5 — external game servers and streams are not reliable
12	Overall cost	Out (constraint)	Shapes how we hit the other targets, isn't a target itself

3. The nine we kept

For each one: where in the kata it comes from, what number we're actually aiming for, and — where we already checked it against our own code yesterday — what we found.

3.1 Partitioning type

Comes from: C6 — “distinct domains ... are built by different team members and should evolve and deploy independently.” That's more or less a direct instruction to partition by domain, not by technical layer.

Target: the distributed version ends up with 5 domain-owned units — gateway, identity, tournament, ingestion, leaderboard — each in its own folder with its own database, no table shared across two of them.

Where we're at: checked this by just reading through the code — each service defines its own SQLAlchemy models and none of them import from another service's models. Holds.

3.2 Simplicity

This one isn't spelled out anywhere in the kata directly, but the assignment itself only makes sense if simplicity counts as a real characteristic: we were asked to build a monolith and a distributed version specifically to compare them, and simplicity is basically the whole reason the monolith exists.

Target: someone who's never seen the code should be able to clone it, run one command, and get through the full flow (register → create a tournament → report a result → see the leaderboard update) in under 15 minutes, using only the README, with no extra infrastructure like a message broker.

Where we're at: got through the whole flow in under 10 minutes on our first real run yesterday, no prior familiarity needed beyond copy-pasting curl commands.

3.3 Modularity

Comes from: C6 and C7 together — independent domains, containerised, meant to scale on demand.

Target: 5 images in the distributed version that each build on their own, with zero compile-time dependency on any of the others.

Where we're at: confirmed — every service folder has its own requirements.txt and Dockerfile, nothing points at a sibling folder.

3.4 Testability

Comes from: R5 (results get validated, disputes get resolved) and R9 (anti-cheat, audit trail).

Target: the one property that actually matters here — a duplicate result should never get counted twice — needs to be checkable with a single repeatable test: report the same match result twice and see if it only counts once.

Where we're at: we actually ran into this for real. After fixing a bug in leaderboard-service (it was crashing on new players because of a None rating), we reported a result and checked the leaderboard — it came back with rating 1010, one win, one loss, exactly once. Automating that as an actual CI test instead of doing it by hand is still on the to-do list.

3.5 Deployability

Comes from: C7 — has to run in containers, locally and in the cloud, no exceptions.

Target: one command, no manual setup steps, under 3 minutes from a clean checkout to a running system, for both architectures.

Where we're at: the monolith was healthy in under a minute; the full 6-container distributed stack (gateway, identity, tournament, ingestion, leaderboard, Redis) took under two.

3.6 Evolvability

Comes from: C6, same as partitioning type, but this is about changing things over time rather than the initial split.

Target: one person should be able to change, rebuild, and redeploy a single service without touching (or restarting) anything else that's running.

Where we're at: this happened by accident in a good way — the leaderboard bug fix was rebuilt and redeployed on its own, identity/tournament/ingestion stayed up and untouched the whole time. Real evidence, not a hypothetical.

3.7 Responsiveness

Comes from: R6 (near-real-time standings), R7 (read-heavy public pages), and C4 (global audience, so latency matters).

Target: leaderboard and bracket reads under 300ms (p95) locally; in the distributed version, a reported result should show up in the leaderboard within 5 seconds even though it travels through Redis first.

Where we're at: in our own testing it showed up in well under a second, but that's one manual test on one laptop, not a proper load test — we're flagging that honestly rather than pretending we measured p95 for real.

3.8 Scalability

Comes from: U8 (accounts expected to go from 50k to 500k over about two years) plus R2 and R4, since bigger tournaments mean bigger brackets.

Target: the distributed system should handle 10x the accounts and 5x the concurrent competitors without a redesign, by letting tournament-service, ingestion-service, and leaderboard-service scale to different numbers of replicas independently.

Where we're at: scaling ingestion-service to 3 replicas works and results still only get processed once, thanks to how Redis consumer groups behave — but we haven't actually thrown anything close to 500k accounts' worth of load at it, so this is designed-for rather than proven.

3.9 Elasticity

Comes from: U10 and C1 — spectator counts jumping to 100k–500k for a few hours during a final, then dropping back down.

Target: leaderboard-service (the read-only path spectators actually hit) should scale from 2 to 10 replicas automatically based on CPU load, then scale back down once the spike passes.

Where we're at: we wrote the HorizontalPodAutoscaler config and applied it locally, but actually simulating a 100k-person burst is out of reach for a course project — that's future work, said plainly rather than dressed up.

3.10 Fault tolerance

Comes from: C2 and C5 — results come from a bunch of external game servers and streaming platforms, and those are exactly the kind of thing that goes down.

Target: if tournament-service (the write side) goes down, leaderboard-service should keep answering GET requests with whatever it last knew, instead of failing too.

Where we're at: tested directly — stopped tournament-service, hit the leaderboard endpoint, it kept working.

4. What we left out, and why

Overall cost isn't really a target you design toward — it's more of a constraint sitting underneath every other decision above. It's the reason we used SQLite instead of Postgres, a background thread instead of a whole separate worker container, and five services instead of fifteen. With two people and a week, keeping the implementation cost down was non-negotiable on every single choice, which is different from it being its own measurable target.

Maintainability overlaps enough with evolvability and testability that treating it separately felt redundant for a project this size — there's no real long-term maintenance horizon here, we're not shipping this to production. That said, the bug fix we did is honestly decent evidence the code is maintainable in practice, even without us setting it as a formal target.

5. Summary

Characteristic	Comes from	Target	Where we're at
Partitioning type	C6	5 domain-owned units, no shared tables	Done
Simplicity	implicit / monolith	≤15 min onboarding, one command, no extra infra	Done
Modularity	C6, C7	5 independently buildable images	Done
Testability	R5, R9	Duplicate-result replay test, checkable via /healthz	Done manually; CI later
Deployability	C7	One command, under 3 min to build and start	Done
Evolvability	C6	Redeploy one service without touching the rest	Done — proved it fixing the leaderboard bug
Responsiveness	R6, R7, C4	<300ms p95 reads, result visible within 5s	Looks fine informally; no real load test yet
Scalability	U8, R2, R4	10x accounts / 5x competitors, independent scaling	Designed for; not load tested at that scale

Elasticity	U10, C1	2→10 replicas on leaderboard-service via HPA	HPA written; never tested with a real burst
Fault tolerance	C2, C5	Leaderboard keeps working if tournament-service dies	Done — tested it directly

Appendix

The updated kata, with every requirement/user/context item numbered (R1–R10, U1–U11, C1–C8) so the references above actually point at something specific, is attached as a separate file alongside this one.